Multitask Training with BERT

Stanford CS224N Default Project

Ruining Luo
School of Computing and Data Science
The University of Hong Kong
ning.l.rn@connect.hku.hk

Abstract

This project consists of completing an implementation of the BERT model and exploring various techniques for improving its performance on multiple sentence-level tasks with multi-task learning, as conducted for the CS 224N final project. The models are fine-tuned for sentiment analysis, paraphrase detection, and semantic textual similarity with three corresponding datasets, the Stanford Sentiment Treebank (SST) dataset, the Quora dataset, and the the SemEval STS Benchmark dataset. The experimental results show that techniques including paired encoding, PCGrad, and gradient clipping are helpful in increasing the overall performance of the model.
This report is formatted as a simplified conference paper to document the methodology and results of an independent study on the course CS 224N.

1 Approach

1.1 Main Approaches

Pretrained BERT This project utilizes pretrained BERT model weights and embeddings from a local BERT model ("bert-base-uncased") (1), and attempts to fine-tune the model to improve its performance on the three tasks. The code of the BERT model is largely given and the implementation is done by filling missing code blocks.

Multitask training Rather than fine-tuning on the three tasks independently, the project aims to improve the performance by training over the three tasks simultaneously. The total loss is the sum of losses on all three tasks as proposed by (2).

$$\mathcal{L}_{total} = \mathcal{L}_{sentiment} + \mathcal{L}_{paraphrase} + \mathcal{L}_{similarity} \tag{1}$$

We also experimented with a version of uncertainty weighted loss introduced by Kendall et al.(3), where log-variance parameters s_{task} are introduced and learned to penalize more uncertain updates.

$$\mathcal{L}_{total} = \sum_{task} s_{task}\mathcal{L}_{task} + s_{task} \tag{2}$$

Oversampling and undersampling Since the Quora dataset is significantly larger than the other two datasets, we experimented with oversampling and undersampling to address the class imbalance(4). In earlier experiments, the number of steps of each epoch is the size of the Quora dataset. At each step, examples are drawn from all three datasets, so the SST and STS datasets are iterated over multiple times. While the accuracy of paraphrase detection rises stably with time, the score of sentiment analysis and semantic textual similarity do not converge properly. This indicates that the Quora dataset dominates the training. Later, the number of steps of each epoch is set to the size of the smallest dataset. It proves that undersampling leads to better convergence and helps prevent domination by one dataset.

PCGrad (Gradient Surgery) To mitigate potential conflicts among tasks in multi-task training, we implemented the PCGrad algorithm proposed by Yu et al.(5). It detects if the gradients $\mathbf{g}_i, \mathbf{g}_j$ of the i-th, j-th tasks have negative dot-product, and projects one gradient onto the normal plane of another gradient:

$$\mathbf{g}_i = \mathbf{g}_i - \frac{\mathbf{g}_i \cdot \mathbf{g}_j}{\|\mathbf{g}_j\|^2} \cdot \mathbf{g}_j \tag{3}$$

Sample weighted PCGrad To address the potential statistical effect of the unbalanced sizes of datasets, we attempted to weigh the PCGrad updates with respect to the sizes of the datasets:

$$\mathbf{g}_{task_i} = \frac{\mathbf{g}_{task_i}}{\text{Size}_{task}} \tag{4}$$

MLP prediction heads The provided code originally utilizes independent linear classifiers for each task. Passing embeddings into MLP layers proves to give better performance.

"Paired encoding" In the original code, for paraphrase detection and semantic textual similarity, two sentences are passed independently to BERT and the prediction is based on the concatenation of two embeddings. We modified the dataloaders to concatenate the sentence tokens with a SEP token in between, and use the result to produce a single embedding for prediction(1). This leads to a substantial increase in performance, especially for semantic textual analysis. As there seems to be no specific name for this method, it is referred to as "paired encoding" in the report.

Cosine similarity and CosineEmbeddingLoss Given that sentence embeddings have an intrinsic notion of cosine similarity, we also experimented with simple cosine similarity and CosineEmbeddingLoss[1] on the SemEval dataset.

1.2 Baseline

When training the first model with undersampling, we found that the performance is similar to earlier oversampling models although it requires much less time to train. Therefore, all later models are trained with undersampling, with the same training time of 50 epochs. For consistency and convenience, we use the model with undersampling and MLP prediction heads as baseline.

2 Experiments

2.1 Data

The datasets employed are default datasets provided by the project. For sentiment analysis, we employed the Stanford Sentiment Treebank (SST) dataset[2](6), which consists of 11,855 single sentences from movie reviews labelled with negative, somewhat negative, neutral, somewhat positive, or positive. For paraphrase detection, we employed the Quora dataset[3], which consists of 404,298 question pairs labelled whether the questions are paraphrases of one another. For semantic textual similarity, we employed the SemEval STS Benchmark dataset[4](7), which consists of 8,528 sentence pairs with similarity scores ranging from 0 (unrelated) to 5 (equivalent).

2.2 Evaluation Method

We utilize prediction accuracy for sentiment analysis and paraphrase detection, and Pearson correlation of the true similarity values against predicted values for semantic textual similarity. The metric for overall performance is the collective performance over the three tasks (as in the project description).

$$\text{Score} = \frac{1}{3} \times \text{Acc}_{\text{sentiment}} + \frac{1}{3} \times \text{Acc}_{\text{paraphrase}} + \frac{1}{3} \times \frac{\text{Corr}_{\text{similarity}} + 1}{2} \tag{5}$$

[1] https://pytorch.org/docs/stable/generated/torch.nn.CosineEmbeddingLoss.html

[2] https://nlp.stanford.edu/sentiment/treebank.html

[3] https://www.quora.com/q/quoradata/First-Quora-Dataset-Release-Question-Pairs

[4] https://aclanthology.org/S13-1004.pdf

2.3 Experimental Details

Training and Hyperparameters All models are trained with the following hyperparameters and undersampling for 50 epochs on a single cloud RTX4090 GPU.

- Learning rate of $1e^{-5}$
- Batch size of 32
- BERT hidden size of 768
- MLP hidden size of 768
- Dropout rate of 0.1

Experimentation process We first conducted multiple experiments with oversampling. The models have worse performance than the vanilla undersampling model1. The vanilla undersampling model is set as the baseline, and the training data of the oversampling models are not included in the report.
Since the model still underperforms in semantic textual similarity, we experimented with cosing similarity loss and CosineEmbeddingLoss, which prove to have no significant impact.
Then, we utilize pairwise encoding to let the attention mechanism to better capture the similarity relationship between sentences. It greatly increases the performance on STS Correlation and also Paragraph Detection Accuracy.
Noticing the oscillation in Sentiment Analysis Accuracy2, we hyphothesized that the unbalanced dataset sizes might be an issue. We also hyphothesized that there are conflicts between updates of different tasks. We implemented PCGrad, sample weighted PCGrad, uncertainty weighted loss, and gradient clipping to mitigate the issues.

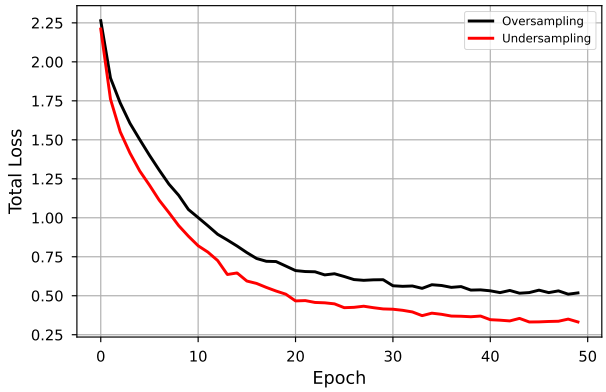
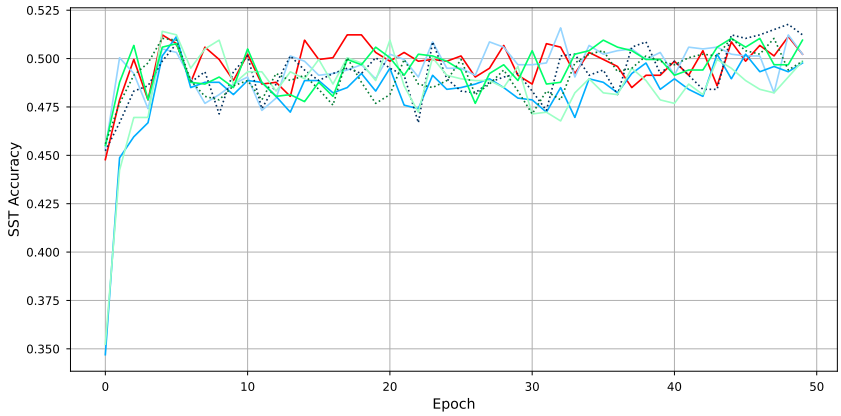


Figure 1: Training loss of vanilla oversampling and undersampling models.



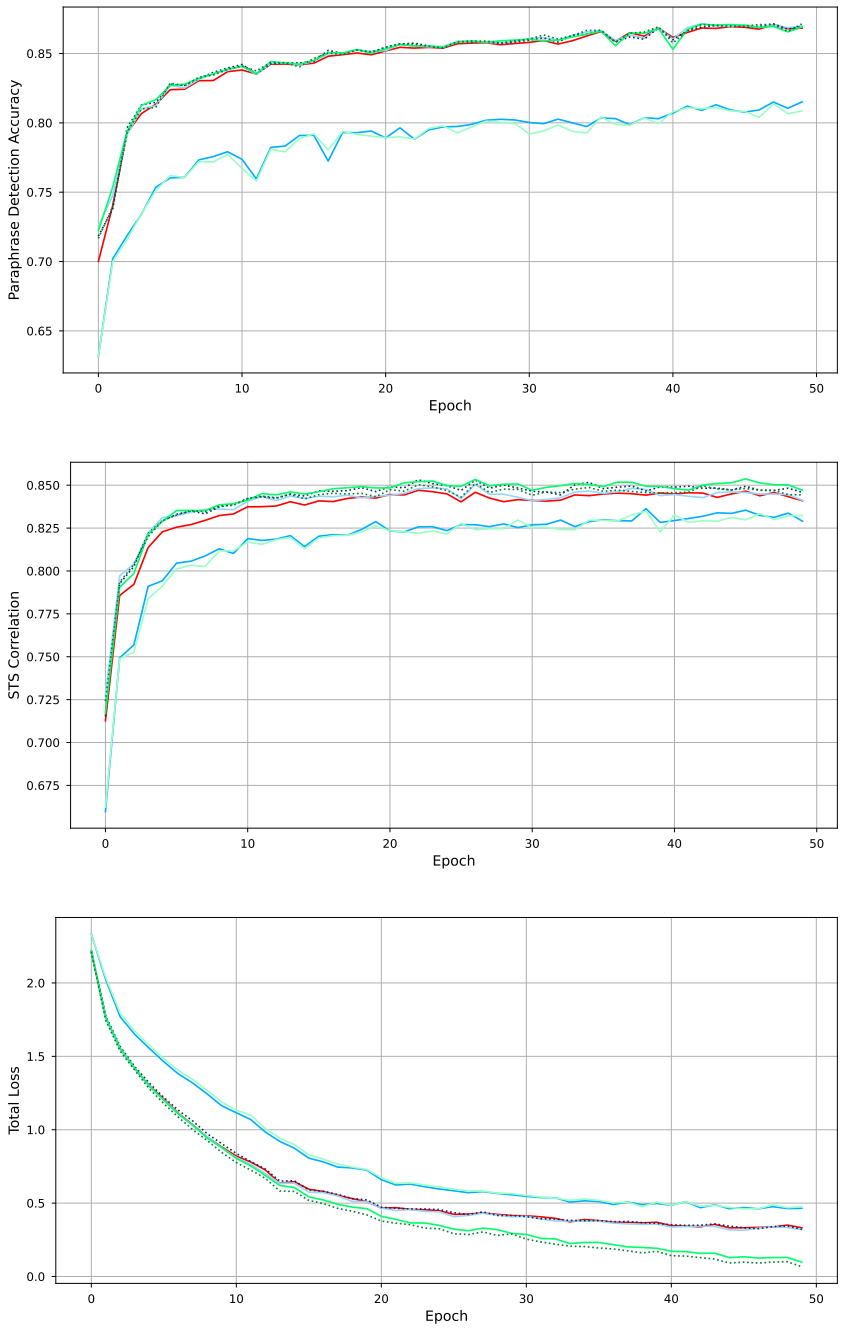


Figure 2: SST accuracy, paraphrase accuracy, STS correlation, and total loss on the development set across 50 epochs. Note that the data of the weighted loss models are not to be directly compared to other data.

2.4 Results

The table below shows the scores of models in the later experiments. While the model with paired

Model	SST Acc	Para Acc	STS Corr	Dev Score
Baseline	0.490	0.748	0.478	0.659
Cosine similarity for STS	0.498	0.747	0.519	0.668
CosineEmbeddingLoss for STS	0.500	0.745	0.395	0.648
Paired encoding	0.501	0.871	0.846	0.765
Paired encoding + PCGrad	0.505	**0.872**	0.843	0.766
Paired encoding + Sample weighted PCGrad	0.498	0.815	0.829	0.742
Paired encoding + PCGrad + Uncertainty weighted loss	0.499	**0.872**	0.844	0.764
Paired encoding + Gradient clipping + PCGrad	**0.515**	0.871	0.847	**0.770**
Paired encoding + Gradient clipping + Sample weighted PCGrad	0.482	0.814	0.830	0.737
Paired encoding + Gradient clipping + PCGrad + Uncertainty weighted loss	0.494	0.871	**0.850**	0.763

encoding, gradient clipping and PCGrad scores highest on overall score and SST Accuracy, the model with paired encoding and PCGrad has the highest Paragraph Detection Accuracy, and the last model has the highest STS Correlation.

3 Analysis

Findings

- **Models trained with oversampling perform worse than those trained with undersampling.** This reveals the negative statistical effect of a much larger dataset, which can be mitigated through random sampling.
- **Paired encoding outperforms cosine similarity and CosineEmbeddingLoss.** It demonstrates the strength of the self-attention mechanism, which is significantly better at capturing the similarities between sentences than simply comparing the embeddings geometrically.
- **Gradient techniques including PCGrad have positive but limited effects.** It might be the case that the three sentence-level tasks are similar and have limited conflicts in updates. The minimal impact of gradient clipping and uncertainty weighted loss suggests that the gradient updates from different datasets are neither excessively large nor highly uncertain.
- **No single best model excels at all three tasks simultaneously.** This indicates that among the techniques explored in our experiments, no definitive solution emerges. Alternative approaches not investigated in this work might hold the key to addressing these challenges.

Limitations

- This project implemented only a limited set of multitask training techniques, leaving numerous potential solutions unexplored.
- The hyperparameters are never tuned in this project. Tuning learning rates and assigning specific weights to the tasks may further help the model to converge on sentiment analysis.

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of*

the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pages 4171–4186, 2019.

[2] Qiwei Bi, Jian Li, Lifeng Shang, Xin Jiang, Qun Liu, and Hanfang Yang. Mtrec: Multi-task learning over bert for news recommendation. In *Findings of the association for computational linguistics: ACL 2022*, pages 2663–2669, 2022.

[3] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7482–7491, 2018.

[4] Roweida Mohammed, Jumanah Rawashdeh, and Malak Abdullah. Machine learning with oversampling and undersampling techniques: overview study and experimental results. In *2020 11th international conference on information and communication systems (ICICS)*, pages 243–248. IEEE, 2020.

[5] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in neural information processing systems*, 33:5824–5836, 2020.

[6] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642, 2013.

[7] Eneko Agirre, Daniel Cer, Mona Diab, Aitor Gonzalez-Agirre, and Weiwei Guo. * sem 2013 shared task: Semantic textual similarity. In *Second joint conference on lexical and computational semantics (* SEM), volume 1: proceedings of the Main conference and the shared task: semantic textual similarity*, pages 32–43, 2013.